


# End-to-End Encrypted Messaging

CS 161 Spring 2026 - Lecture 31

# Last time: Transport-Layer Security (TLS)

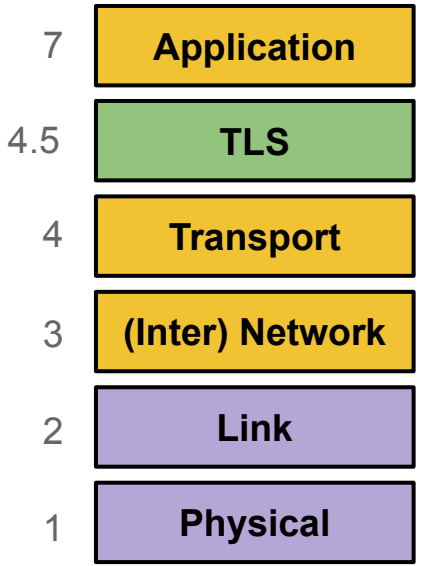


**TLS = End-to-end encryption for any application that runs over TCP**

## Lots of limitations

- Only protects traffic to the host's front-end server
- Doesn't protect communications metadata
- Leaks number of bytes sent
- Relies on brittle CA infrastructure

→ Still, TLS is the best tool we have for this use case

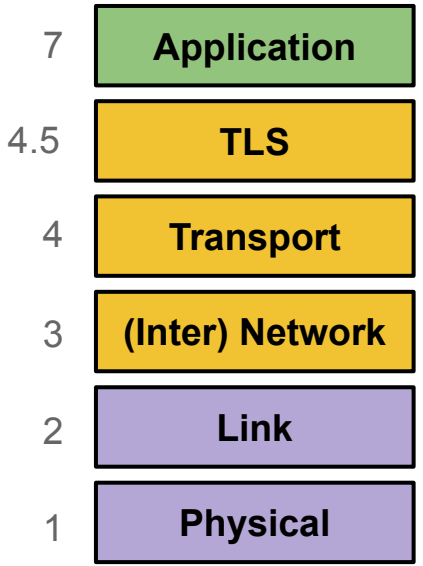


# Today: End-to-end encrypted messaging



- Why study it: A security success story!
  - 20 years ago, unencrypted SMS was the main game in town
  - Now most major messaging apps use end-to-end encryption

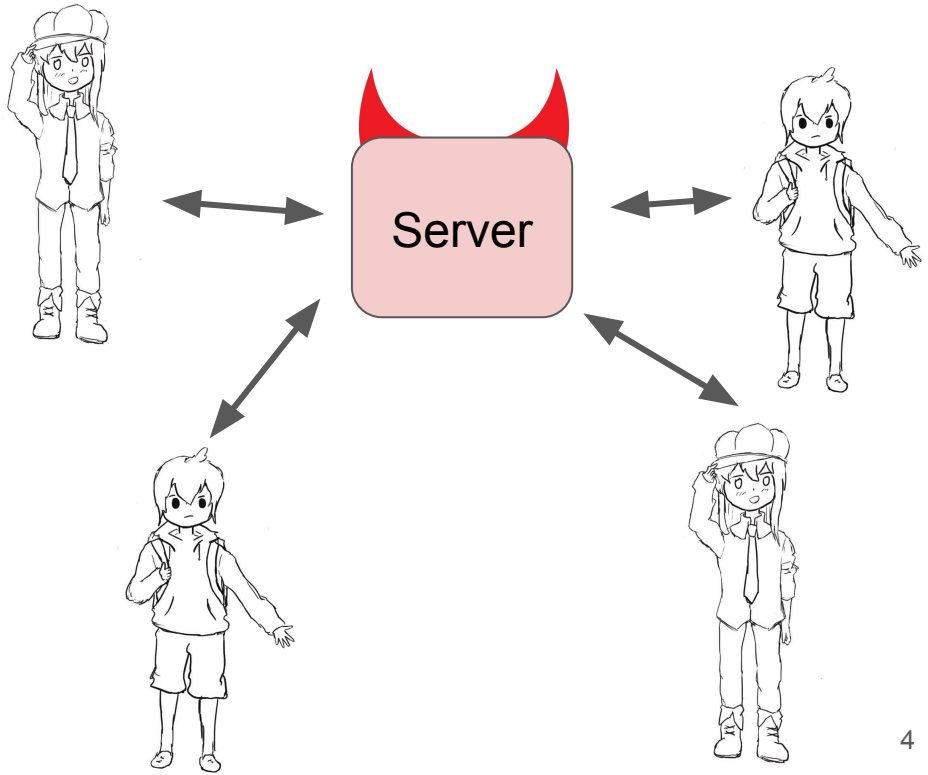
- A high-stakes application
  - Governments
  - Activists
  - Journalists
  - Etc.



# End-to-end encrypted messaging



**Goal:** Clients can exchange messages without revealing to the server what they are saying **even if the server is actively malicious**.




Security only holds if:

- Attacker doesn't control client devices
- Clients have the honest/real version of the messaging app
- The app doesn't have software bugs

(N.B. Nation-state attackers sometimes circumvent encryption by breaking into client devices.)

Updated May 6, 2025 - Technology

# Spyware firm NSO ordered to pay $167M in WhatsApp hacking suit



Sam Sabin

Add Axios on Google




Photo Illustration by Omar Marques/SOPA Images/LightRocket via Getty Images

A California federal jury found that Israel-based spyware vendor NSO Group owes $167.25

https://www.axios.com/2025/05/06/nso-group-whatsapp-jury-damages

# Is this a realistic threat model?



**It seems like we are making contrary assumptions:**

- For security, Signal app must be benign (not backdoored)
- Security goal: Protect us from malicious Signal server
- “If Signal server is malicious, can’t it backdoor the app?”

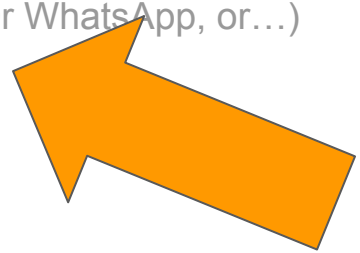
**One view:**

- It’s easy for an attacker to break into server and steal its data
- It’s harder for an attacker to mess around with the code repo
  - This is a “noisy” attack
- E2EE defeats the easy/cheap attacks

# Components of an end-to-end encrypted messenger



**Contact discovery.** Which of your contacts are using Signal? (Or WhatsApp, or…)



**Key lookup.** How do you find your friend's public key?

**Messaging.** How do you encrypt messages to your friend?

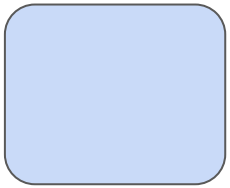
**Encrypted backup.** How can you back up your data without leaking your secret messages?

# Contact discovery









Client

```
(bob, 510-555-1212)
(sam, 510-555-1234)
(kim, 415-555-9921)
        ...
```

Server

```
510-555-1317
510-555-1212
415-555-9921
510-555-1391
510-555-0981
        ...
```

# Contact discovery



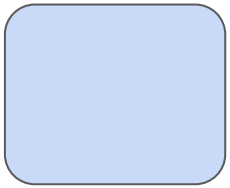


Client

(bob, 510-555-1212)
(sam, 510-555-1234)
(kim, 415-555-9921)
...

Throughout this lecture:
all client-server
communication over TLS





Server

510-555-1317
510-555-1212
415-555-9921
510-555-1391
510-555-0981
...

# Contact Discovery: Trivial protcol



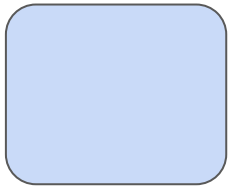


Client

```
(bob, 510-555-1212)
(sam, 510-555-1234)
(kim, 415-555-9921)
      ...
```

```
(bob, 510-555-1212)
(sam, 510-555-1234)
(kim, 415-555-9921)
```

```
555-1212
555-9921
```



Server

```
510-555-1317
510-555-1212
415-555-9921
510-555-1391
510-555-0981
```

**Privacy problem:** Server learns client's entire address book.

# Contact discovery



- A type of “private set intersection” problem
  - Client has: Set *C* (friends’ phone numbers)
  - Server has: Set S (registered users’ phone numbers)
  - Client should learn: $C \cap S$
  - Server should learn: Nothing

- **Privacy for client:** Reveals nothing about her phone book to Signal servers
- **Private for server:** Reveals nothing about other users’ phone numbers

“**Attack**” Alice adds every phone number in the world to her address book

⇒ Can learn entire set of registered users

⇒ Privacy for server only makes sense with **rate limiting**

9TO5Mac

Forums Store Podcasts iPhone Mac iPad Watch Vision Music and TV

PRIVACY SECURITY WHATSAPP META

# WhatsApp security flaw exposed 3.5B phone numbers – including yours [U]

Ben Lovejoy | Nov 18 2025 - 6:39 am PT | 9 Comments



https://9to5mac.com/2025/11/18/whatsapp-security-flaw-exposed-3-5b-phone-numbers-including-yours/

# Implementations of contact discovery



- As far as I can tell, most messengers use “non-private” contact discovery
  - Client sends address book to server
  - Apple, Google Messages, WhatsApp

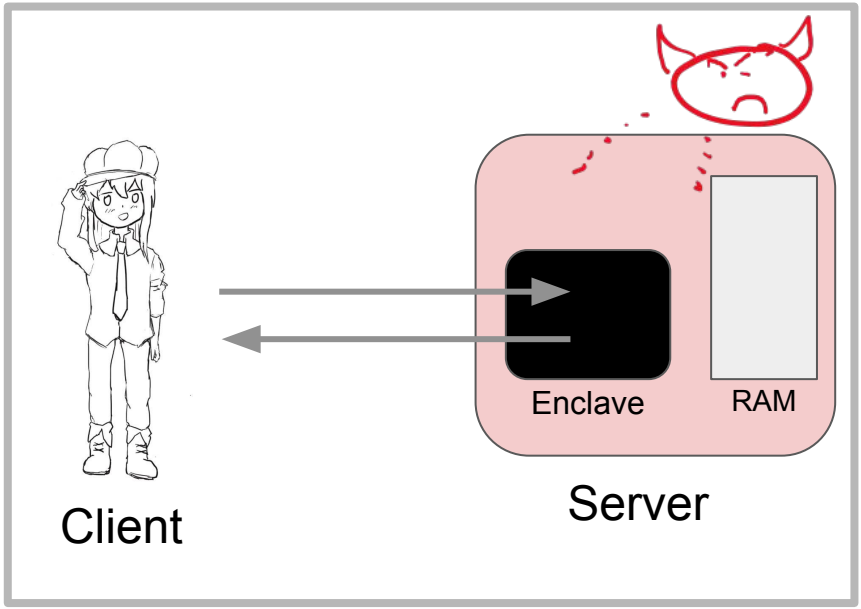
- Signal implemented a more complicated privacy-oriented system
  - Goal: Reveal only the **size** of the client’s address book to the server
  - Requires “trusted execution environment” on the server

- Even Signal server learns a lot about your address book
  - For every contact in your address book:
    - If contact is a Signal user → Signal server learns that they’re your friend
    - If contact is NOT a Signal user → Signal server does not learn that they’re your friend

# Trusted execution environment (TEE) or “hardware enclaves”



- Allows running an x86 program in such a way that even the sysadmin running the server cannot “see into” the program
  - Local variables
  - RAM contents
  - …
- Needs hardware support:
  Intel SGX, AMD SEV, NVIDIA C.C.
- Amazing in principle, brittle in practice
  (see https://sgx.fail/ for a loooong list of attacks)



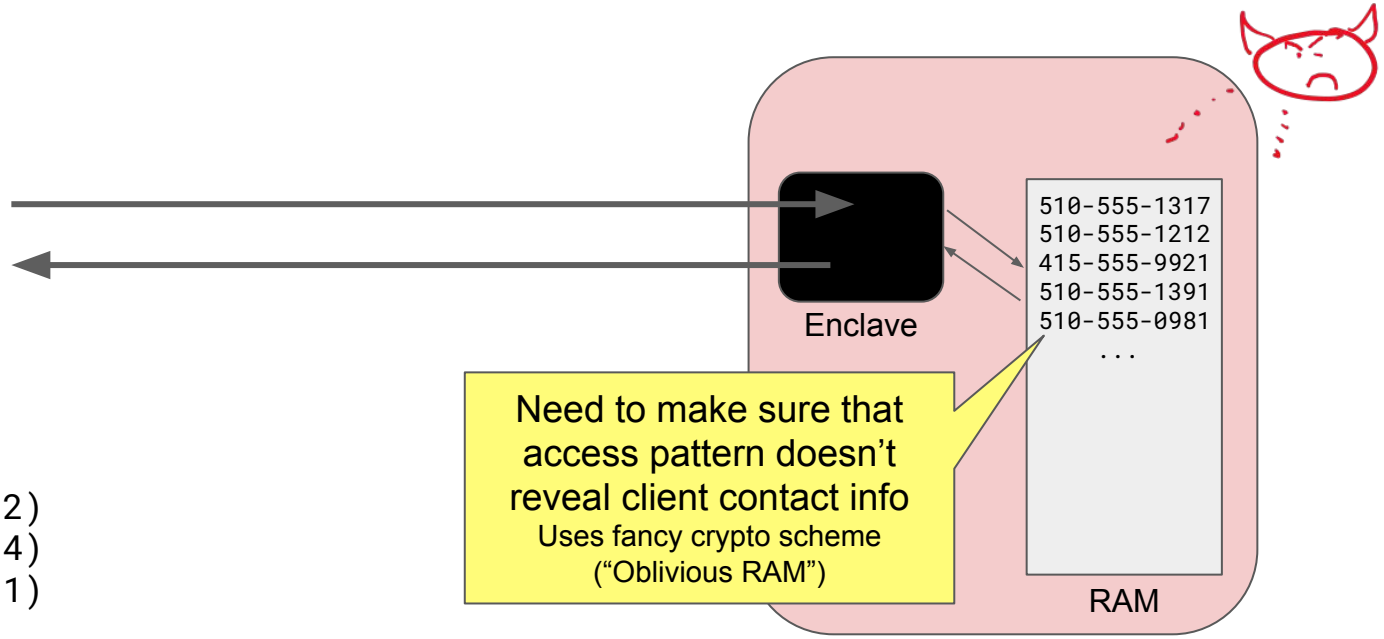
# Signal: Contact discovery with enclaves





Client

```
(bob, 510-555-1212)
(sam, 510-555-1234)
(kim, 415-555-9921)
       ...
```





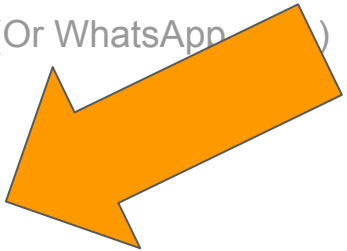

https://signal.org/blog/building-faster-oram/

# Components of an end-to-end encrypted messenger



**Contact discovery.** Which of your contacts are using Signal? (Or WhatsApp…)

**Key lookup.** How do you find your friend's public key?



**Messaging.** How do you encrypt messages to your friend?

**Encrypted backup.** How can you back up your data without leaking your secret messages?

# Key retrieval



**Next step:** For each contact on Signal, fetch contact's long-term public "identity" key

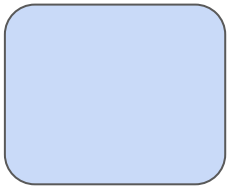
= signature verification public key



Client

```
(bob, 510-555-1212)
(sam, 510-555-1234)
(kim, 415-555-9921)
        ...
```





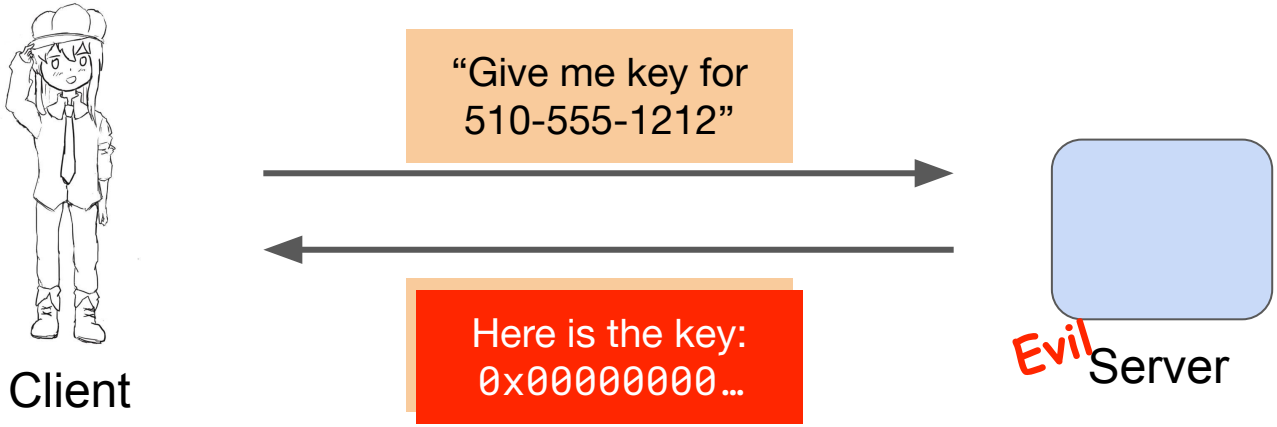
Server

```
(510-555-1212, 0xF169831A…)
(510-555-1234, 0xA8B801DA…)
(415-555-9921, 0xCCC81076…)
        ...
```

# Key retrieval



- Essentially all messaging services do key retrieval in the non-private way




**Privacy problem:** Server learns who you are talking to

**Security problem:** Server could give you the wrong key!

# Key retrieval: Defending against server misbehavior



- Most apps give up on protecting against server compromise proactively.
- Instead: Just make server misbehavior detectable after the fact

Three common defenses:

1) Fingerprints
2) Key-update warnings
3) Transparency logs

# Defense 1: Key fingerprints

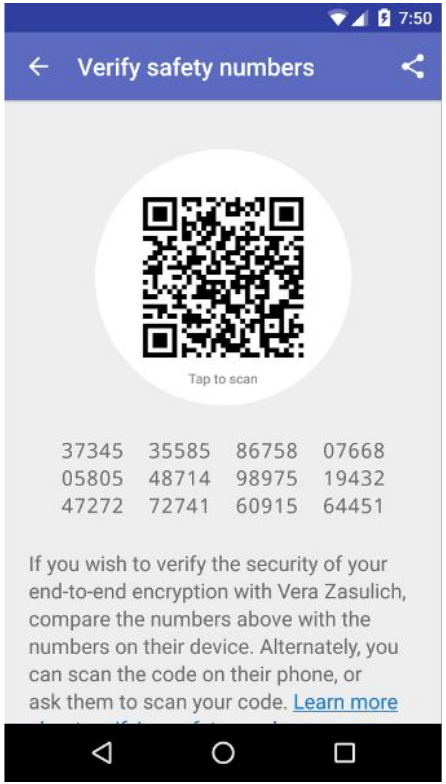


- Show users of messaging app hash of public keys:

  $\text{Hash}(PK_{\text{me}}) \,||\, \text{Hash}(PK_{\text{my-friend}})$

- Users can compare these hashes in person
  - If server tampered with keys, hashes will not match

**Problem:** People rarely actually compare these hashes.




https://signal.org/blog/safety-number-updates/

# Defense 2: Key-update warnings

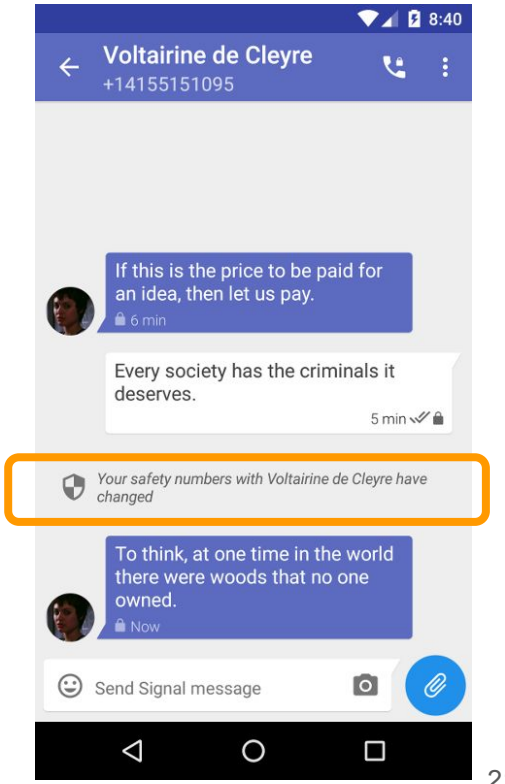


- **Idea:** Assume that the first public key you receive from the server is the correct one.
  - Warn if the key ever changes
  - An instance of “Trust on first use” (TOFU)

- Much nicer for the user than fingerprints
  - Totally transparent in the common case

**Problems:**

- Doesn’t defend against a server that starts out compromised
- False warnings when users reinstall the app/get a new phone




https://signal.org/blog/safety-number-updates/

# Defense 3: Transparency logs (Grossly oversimplified!)

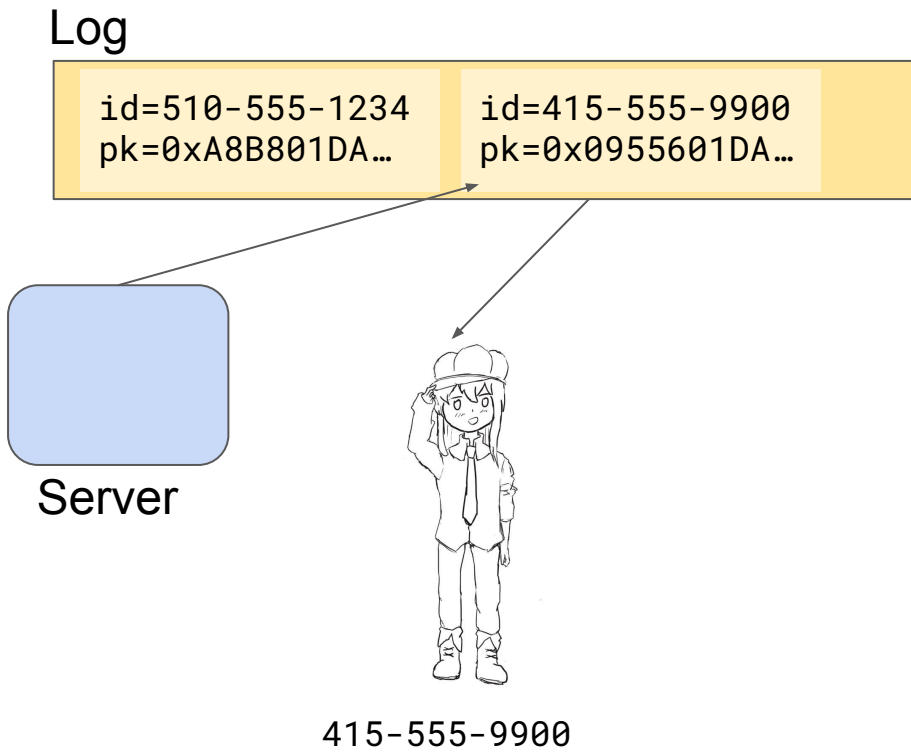


- Server publishes pairs of `(user_id, pk)` to an append-only public log
- Clients can detect for server misbehavior by asking the log for their own public key
- When client asks server for another client's public key, server provides log "inclusion proof"

**Problem:**

- Complicated – Who runs the log?
- Vulnerable unless clients frequently check their keys in the log

→ iMessage and WhatsApp both use this



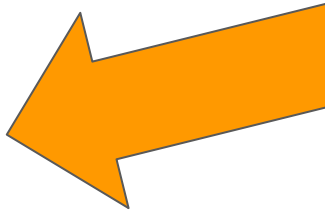
# Components of an end-to-end encrypted messenger



**Contact discovery.** Which of your contacts are using Signal? (Or WhatsApp, or…)

**Key lookup.** How do you find your friend's public key?

**Messaging.** How do you encrypt messages to your friend?



**Encrypted backup.** How can you back up your data without leaking your secret messages?

# Exchanging messages



- The “Signal protocol” is a set of protocols for key exchange and encryption
  - Original developed for the Signal app
  - Now used in WhatsApp, iMessage, Google Messages, …

- As in TLS, there are two main phases:
  - **Handshake**/Key exchange (“X3DH”) – for deriving session shared secret
  - **Record protocol** (“Double ratchet”) – for exchanging messages

- Goals are similar to those of TLS: (also challenging to define precisely!)
  - Confidentiality
  - Integrity
  - “Forward secrecy” = Compromise at time *T* doesn’t reveal keys from times < *T*
  - “Break-in recovery” = Getting a device snapshot at time *T* doesn’t reveal secrets at times >*T*

# Ratcheting to achieve forward secrecy



**Forward security:** If attacker learns today's keys,
it doesn't learn yesterday's decryption keys

(Note: For this to be meaningful, need to delete messages)

## (Simplified) Symmetric-key ratchet

$K_0$ = Initial shared secret between A and B

$K_1 = \text{Hash}(K_0)$ where Hash() is a cryptographic hash fn ("random oracle")

$K_2 = \text{Hash}(K_1)$

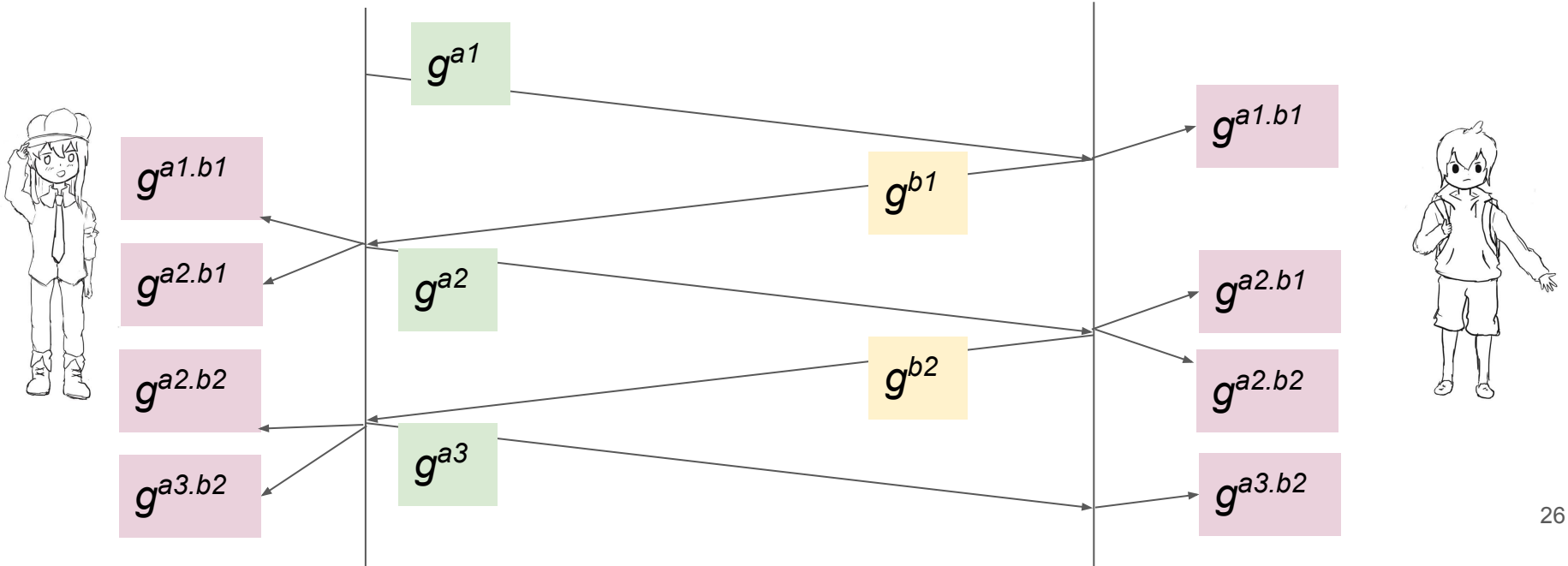
$K_3 = \text{Hash}(K_2)$

– Recovering $K_0$, $K_1$, or $K_2$ from $K_3$ requires inverting the hash function (infeasible).
– BUT, stealing $K_3$ reveals $K_4$, $K_5$, …

# Diffie-Hellman Ratchet



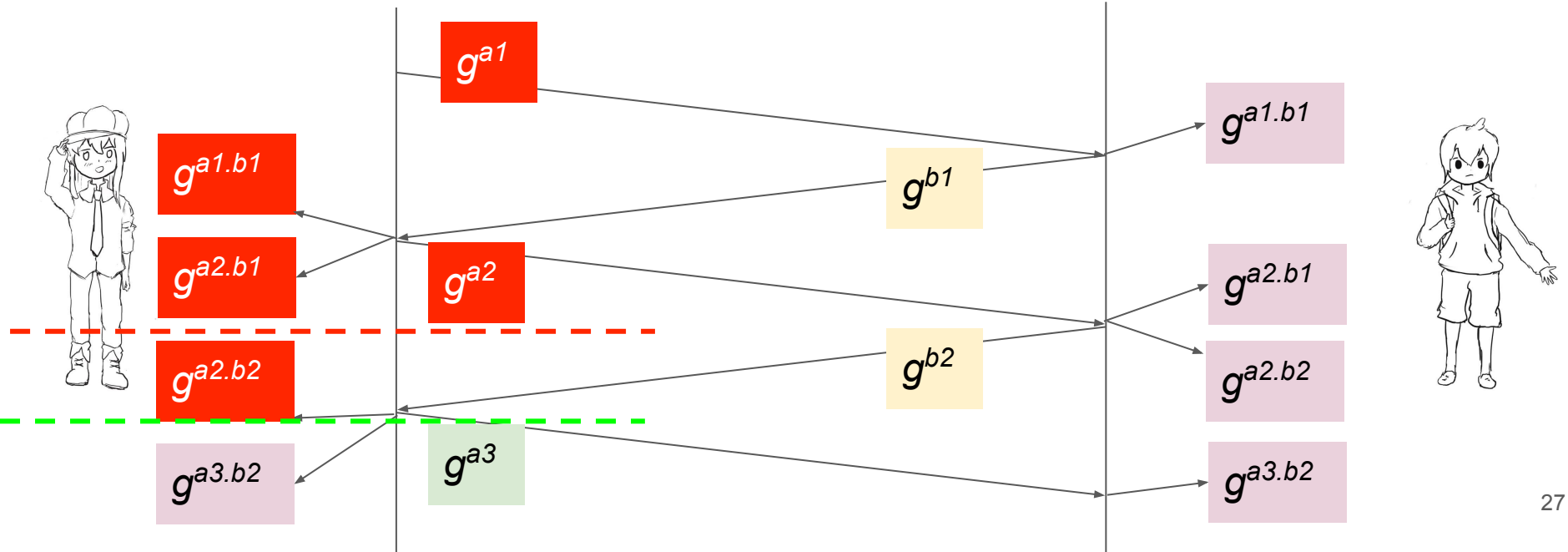
- Client's use a key-exchange ratchet to continuously update their shared secret
  - This provides "break-in recovery" or "healing"



# Diffie-Hellman Ratchet



- Client's use a key-exchange ratchet to continuously update their shared secret
  - This provides "break-in recovery" or "healing"



# Ratcheting in Signal



Signal uses both the symmetric-key and DH ratchet → “Double ratchet”

- **Sym-key ratchet** …on every message sent
- **DH ratchet** …on every message received

→ Only possible because DH keys are short (**32B**) and easy to generate

Post-quantum-secure equivalents of DH have much bigger keys: **800-1500 B**

- New ratcheting schemes under development

**Question:** Why don’t we do per-round-trip ratcheting in TLS?

# Components of an end-to-end encrypted messenger

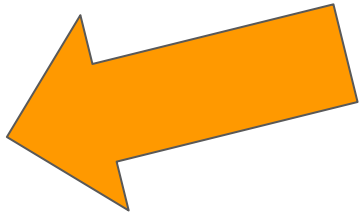


**Contact discovery.** Which of your contacts are using Signal? (Or WhatsApp, or…)

**Key lookup.** How do you find your friend's public key?

**Messaging.** How do you encrypt messages to your friend?

**Encrypted backup.** How can you back up your data without leaking your secret messages?



# Encrypted backup



- Backup should let you recover your data even if you lose your phone
- Backups = Soft underbelly of encrypted messaging apps
- Many platforms back up messages in plaintext to device vendor (Apple, etc.)

# Data categories and encryption

The table below provides more detail on how iCloud protects your data when using standard data protection or Advanced Data Protection.

| | Standard data protection | | Advanced Data Protection | |
|---|---|---|---|---|
| **Data category** | **Encryption** | **Key storage** | **Encryption** | **Key storage** |
| **iCloud Mail (1)** | In transit & on server | Apple | In transit & on server | Apple |
| **Contacts (2)** | In transit & on server | Apple | In transit & on server | Apple |
| **Calendars (2)** | In transit & on server | Apple | In transit & on server | Apple |
| **iCloud Backup (including device and Messages backup) (3)** | In transit & on server | Apple | End-to-end | Trusted devices |
| **iCloud Drive (4)** | In transit & on server | Apple | End-to-end | Trusted devices |
| **Photos** | In transit & on server | Apple | End-to-end | Trusted devices |

https://support.apple.com/en-us/102651

# PIN-based encrypted backup



- **Better idea:** Store backups under encryption
- A common design again uses hardware enclaves
  - Apple, Google, Signal, WhatsApp
- Uses public-key encryption

# Encrypted backup with hardware enclave





Client

$PK_{enclave}$

$ct \leftarrow Enc(PK_{enclave}, (data, PIN))$

ct

$SK_{enclave}$

Enclave

Server



https://signal.org/blog/building-faster-oram/

# Encrypted backup with hardware enclave





Client

$PK_{enclave}$

$SK_{alice}$

$ct' \leftarrow Enc(PK_{enclave}, (PIN||PK_{alice})$

$Enc(PK_{alice}, (data))$

ct

Rate limit, decrypt, check PINs

$SK_{enclave}$

Enclave

Server



https://signal.org/blog/building-faster-oram/

# Summary: End-to-end encrypted messaging



**Goal:** Clients can exchange messages without revealing to the server what they are saying **even if the server is actively malicious**.

**Key tools**

- Contact discovery → Hardware enclaves
- Key lookups → Transparency logs
- Messaging → Key ratcheting
- Backups → Enclaves

**Limitation:** Server still learns who you are talking to and how much you are saying.

